# Connecting to MongoDB over Azure Private Link from Snowflake

Technical guidance for secure access to a MongoDB replica set via Azure Private Link Service — UK South region

## Overview

This document outlines three approaches for connecting Snowflake to a MongoDB replica set over a private network on Azure (UK South). The right choice depends on the specific setup.

The customer already has a similar pattern working for Git access following the private Git connectivity guide. The same foundational approach applies to MongoDB, with some additional considerations for replica set topology.

## Option 1: Outbound Private Connectivity via Azure Private Link (Recommended)

> This is the closest analog to the Git private connectivity pattern. It requires **Business Critical edition** and works when MongoDB is hosted in an Azure VNet.

### How it works

1. **Expose MongoDB as an Azure Private Link Service** — MongoDB isn't a service that Snowflake can connect to directly via a native private endpoint. You need to put a **Standard Load Balancer** in front of the MongoDB replica set in your VNet, then create an **Azure Private Link Service** from that load balancer.
2. **Provision the private endpoint in Snowflake**
3. **Approve the connection** in the Azure Portal
4. **Create network rules and an external access integration**
5. **Use from a Python UDF/procedure** with `pymongo`

### Step-by-step SQL

**1. Get the Snowflake principal to allowlist on the Private Link Service:**

```
SELECT key, value
FROM TABLE(FLATTEN(
  INPUT => PARSE_JSON(SYSTEM$GET_PRIVATELINK_CONFIG())
));
```

Note the `privatelink-account-principal` value — you will need this when approving the connection in the Azure Portal.

**2. Provision the private endpoint:**

```
SELECT SYSTEM$PROVISION_PRIVATELINK_ENDPOINT(
  '/subscriptions/<subscription-id>/resourceGroups/<rg-name>/providers/Microsoft.Netw
  'mongodb.internal.example.com'
);
```

Where the first argument is the Azure resource ID of your Private Link Service, and the second is the hostname you will use to reach the MongoDB replica set.

**3. Approve the connection in the Azure Portal:**

Navigate to your Private Link Service in the Azure Portal, go to **Private endpoint connections**, and approve the pending connection from Snowflake.

**4. Verify the endpoint status:**

```
SELECT SYSTEM$GET_PRIVATELINK_ENDPOINTS_INFO();
```

Wait until the output shows `"status": "Approved"` before proceeding.

**5. Create network rule and external access integration:**

```
CREATE NETWORK RULE mongo_private_rule
  MODE = EGRESS
  TYPE = PRIVATE_HOST_PORT
  VALUE_LIST = ('mongodb.internal.example.com:27017');

CREATE SECRET mongo_creds
  TYPE = GENERIC_STRING
  SECRET_STRING = 'mongodb://user:pass@mongodb.internal.example.com:27017/mydb';

CREATE EXTERNAL ACCESS INTEGRATION mongo_eai
  ALLOWED_NETWORK_RULES = (mongo_private_rule)
```

```
  ALLOWED_AUTHENTICATION_SECRETS = (mongo_creds)
  ENABLED = TRUE;
```

## Key consideration: Replica set discovery

> Each member of the replica set has its own hostname. The MongoDB driver performs server discovery and may try to connect to individual replica members. You need to ensure all replica member hostnames resolve correctly through the private endpoint. This typically means either:
>
> - Using a single DNS entry that the load balancer routes to all members, or
> - Configuring `directConnection=true` to avoid replica set discovery

## Azure architecture (UK South)

The setup involves:

1. Create a **VNet** in UK South with subnets spanning availability zones
2. Ensure MongoDB instances are deployed within the VNet
3. Create a **backend pool** containing the MongoDB instances
4. Create a **Standard Load Balancer** (internal) forwarding to the backend pool on port 27017
5. Create an **Azure Private Link Service** attached to the load balancer's frontend IP
6. In Snowflake, provision the private endpoint using the Private Link Service resource ID
7. Approve the connection in the Azure Portal

See: External Network Access and Private Connectivity on Azure

See also: Manage Private Connectivity Endpoints: Azure

## Option 2: Data Connectivity Proxy (DCP)

> Works across all editions (Standard, Enterprise, Business Critical) and is particularly good for **on-premises** MongoDB or environments where Private Link setup isn't feasible. Currently available for **Openflow** connectors.

### How it works

You deploy a lightweight agent container on your network (near MongoDB). The agent makes a single outbound TLS connection on port 443 to Snowflake — no inbound firewall rules needed. Snowflake routes traffic through the encrypted tunnel.

```
CREATE DATA CONNECTIVITY PROXY my_mongo_dcp ENABLED = TRUE;

SELECT SYSTEM$GENERATE_DATA_CONNECTIVITY_PROXY_BOOTSTRAP_TOKEN(
  'my_mongo_dcp', 7
);
-- Deploy the agent container with this token on a VM
-- in the same VNet that can reach MongoDB

CREATE NETWORK RULE mongo_dcp_rule
  MODE = DATA_CONNECTIVITY_PROXY_EGRESS
  TYPE = HOST_PORT
  VALUE_LIST = (
    'mongo-primary.internal:27017',
    'mongo-secondary1.internal:27017',
    'mongo-secondary2.internal:27017'
  );

CREATE EXTERNAL ACCESS INTEGRATION mongo_dcp_eai
  ALLOWED_NETWORK_RULES = (mongo_dcp_rule)
  ENABLED = TRUE;

ALTER DATA CONNECTIVITY PROXY my_mongo_dcp
  SET EXTERNAL_ACCESS_INTEGRATIONS = (mongo_dcp_eai);
```

### Advantages

- No load balancer or Private Link Service setup required
- Works on-premises and across network boundaries
- No Business Critical requirement

- Zero-trust compatible (outbound only from agent)

## Current limitation

DCP is currently available for use with Openflow; broader support for additional Snowflake services is planned for a future release.

## Option 3: External Access Integration over Public Internet

> The simplest option if MongoDB Atlas or the replica set has a public endpoint (even if IP-restricted). No Private Link required.

```
CREATE NETWORK RULE mongo_rule
  MODE = EGRESS
  TYPE = HOST_PORT
  VALUE_LIST = (
    'cluster0-shard-00-00.mongodb.net:27017',
    'cluster0-shard-00-01.mongodb.net:27017',
    'cluster0-shard-00-02.mongodb.net:27017'
  );

CREATE SECRET mongo_creds
  TYPE = GENERIC_STRING
  SECRET_STRING = 'mongodb+srv://user:pass@cluster0.mongodb.net/...';

CREATE EXTERNAL ACCESS INTEGRATION mongo_eai
  ALLOWED_NETWORK_RULES = (mongo_rule)
  ALLOWED_AUTHENTICATION_SECRETS = (mongo_creds)
  ENABLED = TRUE;
```

Then allowlist Snowflake's egress IPs on the MongoDB side.

## Recommendation Summary

For the described scenario — a **MongoDB replica set** in Azure UK South accessed via **Azure Private Link Service** — **Option 1** is the right approach.

| Consideration | Guidance |
|---|---|
| **Azure setup** | Standard Load Balancer (internal) + Azure Private Link Service in UK South VNet. |
| **Replica set discovery** | Use `directConnection=true` or a single connection string that routes through the load balancer, to avoid the driver trying to connect to individual member hostnames that may not resolve through the private endpoint. |
| **Authentication** | Store MongoDB credentials in a Snowflake SECRET, reference in the EAI. |
| **Edition** | Requires Business Critical. If not available, consider DCP (Option 2) when it supports your use case. |

## Comparison of Options

| Capability | Option 1: Azure Private Link | Option 2: DCP | Option 3: Public |
|---|---|---|---|
| **Traffic path** | Azure backbone (private network) | Encrypted tunnel (outbound 443) | Public internet |
| **Edition required** | Business Critical | Standard+ | Standard+ |
| **Inbound ports** | None from Snowflake side | None | MongoDB must be publicly reachable |
| **On-premises support** | No (Azure VNet only) | Yes | Yes (if public) |
| **Setup complexity** | Medium (LB, Private Link Service, DNS) | Low (deploy agent, bootstrap) | Low |
| **Current availability** | GA for EAI | GA for Openflow; broader planned | GA |

## Key Documentation Links

- External Network Access Overview
- External Network Access and Private Connectivity on Azure
- Manage Private Connectivity Endpoints: Azure
- Data Connectivity Proxy
- Setting Up Private Git Connectivity (existing pattern)
- SYSTEM$PROVISION_PRIVATELINK_ENDPOINT